

Indexer

Question 1 (Basic Indexer)

Create a class MyCollection that stores 3 string values internally.

Tasks:

- Use an indexer to set and get values.
- In Main(), store values "A" and "B" using index.
- Print the first value.

using System;

```
class MyCollection
{
    private string[] data = new string[3];

    public string this[int index]
    {
        get { return data[index]; }
        set { data[index] = value; }
    }
}

class Program
{
    static void Main()
    {
        MyCollection obj = new MyCollection();

        obj[0] = "A";
        obj[1] = "B";

        Console.WriteLine(obj[0]); // Output: A
    }
}
```

Example-2:

```
class Student
{
    private int[] marks = new int[5];

    public int this[int index]
    {
        get
        {
            if (index >= 0 && index < 5)
                return marks[index];
            return -1;
        }
        set
        {
            if (index >= 0 && index < 5)
                marks[index] = value;
        }
    }
}
```

Generics**Question 3 (Generic Class)**

Create a generic class Box<T>.

Tasks:

- Store a value of type T.
- Create a method Show() to print the value.
- In Main(), use it for int and string.

```
using System;
```

```
class Box<T>
{
    public T value;

    public void Show()
    {
        Console.WriteLine(value);
    }
}
```

```
}
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        Box<int> intBox = new Box<int>();
```

```
        intBox.value = 10;
```

```
        intBox.Show();
```

```
        Box<string> strBox = new Box<string>();
```

```
        strBox.value = "Hello";
```

```
        strBox.Show();
```

```
    }
```

```
}
```

Example-2:

```
using System;
```

```
class Program
```

```
{
```

```
    static void Print<T>(T data)
```

```
    {
```

```
        Console.WriteLine(data);
```

```
    }
```

```
    static void Main()
```

```
    {
```

```
        Print<int>(100);
```

```
        Print<string>("C# Generics");
```

```
    }
```

```
}
```

Question 5 (Generic Collection)

Use a generic collection.

Tasks:

- Create a List<int>.
- Add at least two values.
- Print them using a loop.

```
using System;
```

```
using System.Collections.Generic;
```

```
class Program
```

```
{
```

```
    static void Main()
```

```
    {
```

```
        List<int> numbers = new List<int>();
```

```
        numbers.Add(10);
```

```
        numbers.Add(20);
```

```
        foreach (int n in numbers)
```

```
        {
```

```
            Console.WriteLine(n);
```

```
        }
```

```
    }
```

```
}
```

Delegates

Question 6 (Simple Delegate)

Create a delegate that points to a method with no parameters.

Tasks:

- Declare delegate.
- Create method ShowMessage().
- Assign and call delegate.

using System;

```
class Program
{
    // Delegate declaration
    public delegate void MyDelegate();

    static void ShowMessage()
    {
        Console.WriteLine("Hello from Delegate!");
    }

    static void Main()
    {
        MyDelegate d = ShowMessage; // Assign method
        d(); // Call delegate
    }
}
```

Question 7 (Delegate with Return Value)

Create a delegate for a calculator.

Tasks:

- Delegate should take two integers and return int.
- Implement method Add().
- Call delegate and print result.

using System;

```
class Program
{
    public delegate int Calculator(int a, int b);

    static int Add(int x, int y)
    {
        return x + y;
    }

    static void Main()
    {
        Calculator calc = Add;
        Console.WriteLine(calc(5, 3)); // Output: 8
    }
}
```

Question 8 (Multicast Delegate)

Demonstrate multiple methods using one delegate.

Tasks:

- Create delegate Notify.
- Create two methods.
- Add both methods to delegate.
- Call delegate and show both outputs.

using System;

```
class Program
{
    public delegate void Notify();
```

```

static void Method1() => Console.WriteLine("Method 1");
static void Method2() => Console.WriteLine("Method 2");

static void Main()
{
    Notify n = Method1;
    n += Method2; // Add another method

    n(); // Both methods execute
}
}

```

Events

Question 9 (Basic Event)

Design a publisher-subscriber model.

Tasks:

- Create class Publisher with event OnProcessCompleted.
- Trigger event inside a method.
- Create Subscriber class with method to display message.
- Subscribe and execute.

```
using System;
```

```

class Publisher
{
    public delegate void Notify();
    public event Notify OnProcessCompleted;

    public void Process()
    {
        Console.WriteLine("Processing...");
        OnProcessCompleted?.Invoke(); // Trigger event
    }
}

```

```

class Subscriber
{
    public void Message()
    {
        Console.WriteLine("Process Completed!");
    }
}

```

```

    }
}

class Program
{
    static void Main()
    {
        Publisher p = new Publisher();
        Subscriber s = new Subscriber();

        p.OnProcessCompleted += s.Message;
        p.Process();
    }
}

```

Question 10 (Event with Parameter)

Create an alarm system.

Tasks:

- Event should pass a string message.
- Trigger event with message "Fire Alarm!".
- Handle event using lambda expression.

```
using System;
```

```

class Alarm
{
    public delegate void AlarmHandler(string msg);
    public event AlarmHandler OnAlarm;

    public void Trigger()
    {
        OnAlarm?.Invoke("Fire Alarm!");
    }
}

```

```

class Program
{
    static void Main()
    {
        Alarm alarm = new Alarm();
    }
}

```



```

        alarm.OnAlarm += (msg) => Console.WriteLine(msg);

        alarm.Trigger();
    }
}

```

Practice Problems

Example 1: Overloading by Different Data Types

```
using System;
```

```

class Sample
{
    private string[] names = { "Karim", "Rahim", "Abdul" };

    // Indexer with int
    public string this[int index]
    {
        get { return names[index]; }
    }

    // Overloaded indexer with string
    public string this[string name]
    {
        get
        {
            foreach (var n in names)
            {
                if (n == name)
                    return "Found: " + n;
            }
            return "Not Found";
        }
    }
}

```

```

class Program
{
    static void Main()
    {
        Sample s = new Sample();
    }
}

```

```

        Console.WriteLine(s[1]);    // Access by index
        Console.WriteLine(s["Bob"]); // Access by name
    }
}

```

Example 2: Indexer with Multiple Parameters (2D Access)

```

using System;
class Matrix
{
    private int[,] data = new int[2, 2];

    public int this[int row, int col]
    {
        get { return data[row, col]; }
        set { data[row, col] = value; }
    }
}

class Program
{
    static void Main()
    {
        Matrix m = new Matrix();

        m[0, 0] = 10;
        m[1, 1] = 20;

        Console.WriteLine(m[0, 0]);
        Console.WriteLine(m[1, 1]);
    }
}

```

Example 3: Overloading by Number of Parameters

```

using System;

class DataStore
{
    private int[] arr = { 10, 20, 30 };

    // Single parameter indexer
    public int this[int index]
    {

```

```

        get { return arr[index]; }
    }

    // Overloaded indexer with two parameters
    public int this[int index, bool square]
    {
        get
        {
            if (square)
                return arr[index] * arr[index];
            else
                return arr[index];
        }
    }
}

class Program
{
    static void Main()
    {
        DataStore d = new DataStore();

        Console.WriteLine(d[1]);        // normal
        Console.WriteLine(d[1, true]);   // squared
    }
}

```

Example-4:

```

using System;
class SampleCollection<T>
{
    private T[] arr = new T[100];

    public T this[int i]
    {
        get => arr[i];
        set => arr[i] = value;
    }
}

class Program
{
    static void Main()

```

```

{
    var col = new SampleCollection<string>();

    col[0] = "Hello";
    col[1] = "World";

    Console.WriteLine(col[0]);
    Console.WriteLine(col[1]);
}
}

```

Example-5:

```
using System;
```

```

abstract class Shape
{
    public abstract double Area();
}

class Circle : Shape
{
    public double Radius;

    public Circle(double r)
    {
        Radius = r;
    }

    public override double Area()
    {
        return 3.1416 * Radius * Radius;
    }
}

class Rectangle : Shape
{
    public double Length;
    public double Width;

    public Rectangle(double l, double w)
    {
        Length = l;
        Width = w;
    }
}

```

```

    }

    public override double Area()
    {
        return Length * Width;
    }
}

class Program
{
    static void Main()
    {
        Shape s1 = new Circle(5);
        Shape s2 = new Rectangle(4, 6);

        Console.WriteLine("Circle Area: " + s1.Area());
        Console.WriteLine("Rectangle Area: " + s2.Area());
    }
}

```

Example-6:

```

using System;

interface IPayment
{
    void Pay(double amount);
}

class BkashPayment : IPayment
{
    public void Pay(double amount)
    {
        Console.WriteLine("Paid " + amount + " using Bkash.");
    }
}

class CardPayment : IPayment
{
    public void Pay(double amount)
    {
        Console.WriteLine("Paid " + amount + " using Card.");
    }
}

```

```
class Program
{
    static void Main()
    {
        IPayment p1 = new BkashPayment();
        IPayment p2 = new CardPayment();

        p1.Pay(500);
        p2.Pay(1000);
    }
}
```